

PLAN DE COURS

Maintenance de logiciel

Titre du cours

Techniques de l'informatique

Nom du ou des programme(s) ou de la composante de formation générale

Technique de l'informatique

Discipline

420-515-MV	1-2-1	1,33
------------	-------	------

Numéro du cours

1-2-1

Pondération

1,33

Unités

Samuel Fostiné	S-012-	samuel.fostine@cegepmv.ca
----------------	--------	--

Enseignant-e

S-012-

Numéro de bureau

samuel.fostine@cegepmv.ca

Poste téléphonique et courriel

Informatique	Olivier Tardif
--------------	----------------

Nom du département

Olivier Tardif

Nom du coordonnateur ou de la
coordonnatrice du département

2026-2027	Automne 2026
-----------	--------------

Année scolaire

Automne 2026

Trimestre

PRÉSENTATION GÉNÉRALE DU COURS :

Ce cours est le dernier dans l'axe de formation *Génie logiciel*, qui contient aussi les cours *Introduction aux bases de données*, *Architecture de logiciel* et *Développement d'applications pour entreprises*. Il est également le dernier cours dans l'axe *Objets connectés*, qui contient aussi les cours *Programmation de plateformes embarquées* et *Introduction aux plateformes IdO*.

Dans ce cours, la personne étudiante est introduite à la plus longue phase du processus de développement: la maintenance. Pendant le reste de son parcours, elle a souvent réalisé des projets pour lesquels elle était responsable de l'écriture de chaque ligne de code. Ici, elle doit lire et comprendre des programmes écrits par d'autres personnes et être capable de les modifier pour ajouter certaines fonctionnalités ou corriger des erreurs, le tout sans briser des fonctionnalités existantes et sans ajouter de nouvelles erreurs. Elle doit aussi déployer ses changements en minimisant les interruptions de service.

Dès le début de la session, une application existante (« legacy »), comportant des bogues, une dette technique et une API à faire évoluer, est fournie par la personne enseignante et sert de fil conducteur pour une partie de la session: chaque semaine, la personne étudiante doit lire, comprendre, déboguer, réusiner (refactoring) et faire évoluer ce code, en plus d'y ajouter des fonctionnalités. Dans une perspective de maintenance à long terme, elle apprend aussi à déployer ses changements en minimisant les interruptions de service, à l'aide d'une plateforme d'intégration et de déploiement en continu (CI/CD) montée avec des outils largement utilisés en industrie : Git, Maven, GitHub Actions et/ou GitLab CI, Docker, un registre de conteneurs (GitHub Container Registry ou GitLab Container Registry), SonarQube/SonarCloud, Trivy, Kubernetes et Helm. Ces outils ne sont pas une fin en soi : ils sont le véhicule qui permet de livrer, de façon sécuritaire et sans interruption, le travail de maintenance réalisé sur le code existant. Le déploiement Kubernetes se fait selon un modèle hybride : un cluster autohébergé au collège, accessible seulement sur place pour des raisons de sécurité réseau, et un cluster Azure Kubernetes Service (AKS) utilisable à distance.

Cette expérience la préparera pour le cours *Stage d'intégration en entreprise*, les développeurs juniors étant souvent attirés à la maintenance.

ÉNONCÉS DE COMPÉTENCE :

Certains éléments des compétences OOSR, OOSV et OOSX seront abordés dans ce cours. Les énoncés de ces compétences sont les suivants :

- La compétence sollicitée dans le cours est la compétence OOSR : *Effectuer le développement d'application natives sans base de données*.
- La compétence sollicitée dans le cours est la compétence OOSV : *Effectuer le développement de services d'échange de données*.
- La compétence sollicitée dans le cours est la compétence OOSX : *Effectuer le développement d'applications pour des objets connectés*.

CIBLE D'APPRENTISSAGE :

- Concevoir et implémenter une mise à jour pour une application existante.
- Créer un environnement d'intégration et de déploiement en continu.
- Développer les réflexes de chaque étudiant pour se préparer à configurer et à maintenir un environnement se rapprochant de celui du marché du travail.

ÉVALUATIONS INTERMÉDIAIRES - QUIZ (15%):

7 quiz concernant la matière vue en classe, seulement les 5 meilleurs quiz seront comptabilisés. Chaque quiz compte pour 3% de la note finale.

Les évaluations sommatives sont notées et consistent en des contrôles pratiques et ou théoriques de courte durée (30 minutes ou moins) visant à évaluer les compétences acquises et la compréhension des notions qui s'y rattachent. Elles ont lieu en classe sous la supervision du professeur et sont individuelles.

ÉVALUATIONS EN TRAVAUX PRATIQUES (50%):

2 travaux pratiques de 10% chaque, puis 2 autres travaux pratiques de 15% chaque.

Ils visent à évaluer les compétences techniques acquises en complétant des tâches précises pour mettre en place des systèmes ou des corrections sur un système donné. Les travaux sont commentés afin d'aider l'étudiant à progresser sur les points à améliorer dans ses méthodes de travail. Certains travaux consisteront à appliquer les étapes pour la création de pipelines et le déploiement d'une application.

ÉVALUATION FINALE (35%):

À partir d'une application déjà conçue et/ou d'un cahier de charge fournis par la personne enseignante, la personne étudiante doit, en équipe, concevoir et déployer une mise à jour du logiciel afin de corriger des bogues, améliorer la structure du code (réusinage) et ajouter des fonctionnalités, en conformité avec le cahier de charge. La mise à jour doit être livrée par un pipeline CI/CD complet (GitHub Actions et/ou GitLab CI, tests automatisés, analyse de qualité et de sécurité du code, conteneurisation, déploiement sur Kubernetes avec Helm – sur AKS ou sur le cluster autohébergé du collège) qui minimise les interruptions de service (déploiement progressif). L'épreuve se réalise sur les trois dernières semaines de la session (semaines 13 à 15), en partie hors des heures de cours et en partie dans des séances de travail supervisé en laboratoire.

TABLEAU DES ÉVALUATIONS

Type d'épreuve	Pondération
7 quiz de révision	15% (3% chacun – les 5 meilleures notes)
4 Travaux pratiques	50% en tout 10% pour les deux premiers TP (chacun) 15% pour les deux derniers TP (chacun)
Examen final	35%

CRITÈRES D'ÉVALUATION:

- Respect rigoureux des consignes
 - La mise en place des solutions doit être fait avec les technologies visées par le cahier de charge. Toute forme d'initiative doit être validé par le professeur.
- Capacité à lire, comprendre et documenter un programme écrit par une autre personne avant de le modifier (individuel).
- Organisation logique des instructions et lisibilité du code.
- Contrôle rigoureux de la qualité de l'application avec l'aide de tests automatisés (JUnit, Cucumber) et d'outils d'analyse de la qualité et de la sécurité du code (SonarQube/SonarCloud, Trivy), intégrés au pipeline CI/CD (GitHub Actions et/ou GitLab CI).
- Repérage complet des erreurs, fonctionnement correct du programme et des composantes électroniques.

CONTENUS ESSENTIELS:

- Phase de maintenance
 - a. Différentes étapes de la maintenance.
 - b. Lecture et compréhension d'un programme écrit par une autre personne (documentation, tests existants, débogueur, exploration progressive du code).
 - c. Importance de la maintenance dans la vie du logiciel.
 - d. Retour sur les pratiques de design facilitant la maintenance. Waterfall vs Agile.
 - e. Fin de vie du logiciel
- Techniques de débogage.
- Techniques de réusinage (refactoring).
- Modification d'API
 - a. Codification de la numérotation des versions
 - b. Processus pour modifier un API côté client/côté serveur
 - c. Stratégies pour éviter la modification d'API
 - d. Bonnes pratiques de la modification d'API
- Méthode de travail (workflow) pour le contrôle/gestion des versions
 - e. Version quotidienne vs stable (Nightly vs stable)

-
- f. Outils contribuant à la méthode de travail pour une plateforme donnée (ex : GitHub/GitLab, GitHub Actions et/ou GitLab CI, Maven, Docker, Kubernetes (AKS et cluster autohbergé), Helm, SonarQube, Trivy, etc.)
 - Survol des outils de gestion des erreurs/bogues
 - Stratégies de déploiement minimisant les interruptions de service (rolling update, blue-green, canary).
 - Distribution de mises à jour
 - a. Sur le matériel informatique
 - b. Par l'entremise de l'internet
 - c. Forcer une mise à jour
 - d. Rétroportage (backporting) des mises à jour (ex : pour la sécurité)
 - e. Accepter les mises à jour pour un objet connecté de façon sécuritaire (ex: signatures avec clés GPG)

STRATÉGIES D'APPRENTISSAGE :

Les activités pratiques sont privilégiées. L'objectif est d'habituer la personne étudiante à travailler avec des programmes existants, à les comprendre et à les modifier. Une application existante, comportant des bogues, une dette technique et une API à faire évoluer, est fournie dès le début de la session et sert de fil conducteur : chaque outil ou technique enseigné (débugage, tests automatisés, réusinage, Git, Maven, GitHub Actions et/ou GitLab CI, Docker, Kubernetes, etc.) est immédiatement appliqué sur ce code réel plutôt qu'enseigné de façon isolée. Le contenu de chaque semaine est calibré pour la pondération réelle du cours (1-2-1, soit 3 heures de cours par semaine) et laisse le temps nécessaire aux quiz et aux contrôles pratiques prévus à l'horaire, sur 13 semaines d'enseignement suivies de deux semaines consacrées à l'épreuve finale.

- Pratique guidée
- Pratique autonome

Le réusinage et à l'extension de programmes fournis par la personne enseignante sont également ciblés.

L'enseignement magistral des concepts et des démonstrations pratiques des outils sont utilisés pour transmettre les contenus essentiels.

HABILETÉS ET ATTITUDES DÉVELOPPÉES :

Les attitudes véhiculées dans ce cours sont celles prévues au profil de sortie, soit :

- ✓ La communication et la collaboration
- ✓ L'esprit critique
- ✓ L'efficacité
- ✓ La polyvalence et l'autonomie
- ✓ La rigueur et l'intégrité

CALENDRIER SYNTHÈSE

Semaine d'enseignement	Nature et date de remise des évaluations	Points alloués	Autres informations (s'il y a lieu)
Semaine 1	• Introduction à la maintenance logicielle (types, cycle de vie, Waterfall vs Agile). • Remise de l'application legacy (fournie par l'enseignant, avec bogues et dette technique). • Git 1/2 : configuration, commit, push, pull, branches.		
Semaine 2	• Lecture et compréhension d'un programme existant (documentation, tests, exploration progressive). • Techniques de débogage 1/2 : reproduction et isolation d'un bogue, lecture de la pile d'appels (stack trace), hypothèses de cause. • Git 2/2 : Pull Request, Merge, Rollback, Cherry-pick.	3	Quiz 1
Semaine 3	• Techniques de débogage 2/2 : • Maven 1/2 : pom.xml, dépendances, plugins.	10	TP1
Semaine 4	• Maven 2/2 : cycle de build, exécution des tests avec Maven. • Introduction aux tests automatisés (JUnit) : premiers tests unitaires sur l'application legacy.	3	Quiz 2
Semaine 5	• Tests automatisés : tests unitaires vs tests de régression, couverture de code, bonnes pratiques d'écriture de tests. • Mise en place d'un premier pipeline d'intégration continue (build + exécution des tests) avec GitHub Actions et/ou GitLab CI	-	Chaque équipe ajoute une suite de tests à l'application legacy et l'intègre au pipeline.

CALENDRIER SYNTHÈSE

Semaine d'enseignement	Nature et date de remise des évaluations	Points alloués	Autres informations (s'il y a lieu)
Semaine 6	• Réusinage (refactoring) et code smells : principes et techniques courantes. • Application d'un réusinage sur un module de l'application legacy, appuyé par la suite de tests existante.	10	TP2
Semaine 7	• Conteneurisation avec Docker (Dockerfile, image, bonnes pratiques). • Intégration de la construction et de la publication de l'image Docker dans le pipeline CI/CD, vers un registre de conteneurs (GitHub Container Registry ou GitLab Container Registry).	3	Quiz 3
Semaine 8	• Évolution d'API : versionnage (SemVer) et compatibilité client/serveur. • Ajout d'une fonctionnalité qui fait évoluer l'API sans casser la compatibilité.	3	Quiz 4
Semaine 9	• Qualité et sécurité du code : analyse statique avec SonarQube/SonarCloud et analyse de vulnérabilités avec Trivy, intégrées au pipeline CI/CD. • Correction des enjeux de qualité et de sécurité détectés sur l'application legacy.	15	TP3
Semaine 10	• Distribution de mises à jour et sécurité (objets connectés, rétroportage de correctifs, mises à jour signées). • Survol des outils de gestion des bogues (GitHub Issues / GitLab Issues).	3	Quiz 5
Semaine 11	• Kubernetes 1/2 : namespace, Deployment, Service, répartition de charge.	3	Quiz 6

CALENDRIER SYNTHÈSE

Semaine d'enseignement	Nature et date de remise des évaluations	Points alloués	Autres informations (s'il y a lieu)
	Architecture hybride : cluster autohébergé au collège (accessible seulement sur place, sans VPN) et cluster AKS (crédit étudiant Azure) pour le travail à distance.		
Semaine 12	- Kubernetes 2/2 : ConfigMap, Secrets, logs, Ingress. - Introduction à Helm et stratégies de déploiement sans interruption (rolling update).	15	TP4 : déploiement de la mise à jour via Helm, avec rolling update. Réalisable sur AKS ou en classe sur le cluster du collège.
Semaine 13	- Révision générale et consolidation des outils du pipeline CI/CD. L'intelligence artificielle en maintenance logicielle : assistants de code, limites et validation humaine obligatoire du résultat. - Remise du cahier de charge de l'épreuve finale : lecture et analyse en équipe.	3	Quiz 7
Semaine 14	- Épreuve finale (1/2) : travail d'équipe supervisé en laboratoire. - Conception et implémentation de la mise à jour du logiciel (réusinage ciblé, correction de bogues, ajout d'une fonctionnalité, suite de tests de régression). - L'utilisation d'outils d'IA est permise, à condition de pouvoir expliquer et justifier tout code produit.	-	Séance de travail supervisé; une partie du travail se poursuit hors des heures de cours.
Semaine 15	- Épreuve finale (2/2) : remise de la mise à jour du logiciel, déployée via le pipeline complet, sans interruption de service. - Examen théorique sur l'ensemble de la matière	35	20% pour la mise à jour du logiciel et son déploiement. 15% pour l'examen théorique.

<u>CALENDRIER SYNTHÈSE</u>			
Semaine d'enseignement	Nature et date de remise des évaluations	Points alloués	Autres informations (s'il y a lieu)
	du cours.		

Les contenus à voir chaque semaine sont donnés à titre indicatif seulement. Ils peuvent changer en fonction du rythme du groupe.

EXIGENCES PARTICULIÈRES DU COURS :

RÈGLES INSTITUTIONNELLES ET DÉPARTEMENTALES :

Usage du cellulaire et appareils électroniques en classe

Dans les lieux d'enseignement, l'utilisation d'ordinateurs portables et d'appareils électroniques (téléphones cellulaires, téléavertisseurs, lecteurs audionumériques, agendas électroniques, caméras numériques, assistants numériques personnels etc.) est interdite. Tout contrevenant pourra être expulsé sans préavis. Ces appareils doivent être rangés hors de vue pour toute la durée des séances de cours.

Enregistrement vocal ou vidéo

Par ailleurs, les usagers de tels appareils doivent respecter l'intégrité physique et morale des personnes. En conséquence, en tout temps et en tous lieux, il est formellement interdit d'enregistrer, de photographier ou de filmer sans le consentement des individus concernés.

Les modalités d'application de la Politique institutionnelle d'évaluation de l'apprentissage (PIEA) sont rendues disponibles aux étudiants et il appartient à ceux-ci d'en prendre connaissance.

Les articles ici-bas qui font l'objet de modalités particulières d'application font référence à ceux de la PIEA en vigueur disponible sur le portail du Cégep Marin-Victorin.

Présence aux évaluations sommatives

Conformément à l'article 4.4.1 de la PIEA :

La présence à une évaluation sommative est obligatoire. L'étudiant qui s'absente, sans motif grave à l'appui, reçoit la note zéro. C'est à l'étudiant qu'il revient d'aviser son professeur des motifs de son absence dans le plus bref délai et de lui fournir, s'il y a lieu, une pièce justificative. Seul un motif grave (ex. mortalité, accident ou maladie) peut être reconnu comme valable par le professeur. Dans un tel cas, selon la nature de l'évaluation, le professeur proposera à l'étudiant une modalité de récupération.

Lors d'un examen, l'étudiant doit se présenter au moment et à l'endroit prévus. S'il arrive en retard et qu'un autre étudiant a déjà terminé et quitté la salle, l'accès lui est refusé, à moins que la nature de l'évaluation le permette.

Remise des travaux

Conformément avec l'article 4.4.2 de la PIEA :

Dans le cas d'un travail, le professeur détermine les modalités de remise, à savoir le lieu et le support (version électronique, version imprimée ou document original). Tout travail qui ne respecte pas ces modalités pourra être refusé. Le professeur détermine également la date et le moment de la remise du travail. L'étudiant qui remet son travail en retard se verra, sauf dans des situations jugées exceptionnelles par le professeur, attribuer une pénalité de 10% de la pondération prévue au départ de ce travail, par jour ouvrable, à compter du jour et de l'heure de la remise du travail.

Par ailleurs, un travail qui n'est pas remis à temps peut être refusé à compter du moment où le professeur utilisera le contenu de ce travail dans le cadre de son cours, ou qu'il sera requis pour poursuivre un travail en équipe. Une telle condition pédagogique doit être indiquée à l'avance aux étudiants, avec les consignes du travail.

Tout travail remis au professeur après que les étudiants ont reçu leurs travaux corrigés est refusé. Seul le professeur, s'il le juge à propos, peut proposer un autre travail et accorder un délai.

Dans tous les cas où le type de travail le permet, l'étudiant doit conserver un brouillon, un fichier électronique ou une photocopie de son travail.

Correction du Français

Conformément avec l'article 4.6.2 de la PIEA :

Dans les productions écrites (examens, travaux, projets), la correction du français est obligatoire et elle constitue une pénalité jusqu'à concurrence de 10% de la note. Pour établir cette pénalité, les productions écrites sont corrigées à l'aide d'une grille à échelle descriptive, selon le type de travail exigé.

Présence en classe

Conformément à l'article 4.7.1 de la PIEA, il appartient à l'étudiant :

- De fournir les efforts nécessaires pour atteindre les objectifs du cours.
- D'être présent, à l'heure, à toutes les périodes de cours prévues à son horaire et d'y participer activement. À défaut d'être présent, il doit récupérer par lui-même les apprentissages manqués.
- De respecter l'horaire prévu de même que le temps de pause. L'étudiant qui ne respecte pas ces exigences pourra subir les sanctions prévues au Règlement relatif aux conditions de vie au Cégep Marie Victorin (Règlement numéro 9).
- De respecter les délais de remise des travaux ou, si cela est impossible, il a la responsabilité d'entrer en contact avec son professeur dans les meilleurs délais (l'article 4.4.2 de la PIEA).

Il appartient aussi à l'étudiant:

- De respecter toutes les autres règles prévues à la politique relative à l'utilisation des technologies de l'information et de la communication
- D'utiliser un langage approprié, courtois et professionnel dans ses communications numériques

De plus, il est strictement interdit de boire ou manger dans les laboratoires.

Conformément avec l'article 4.7.4 de la PIEA :

L'observation progressive des apprentissages

Dans les cours où le travail étudiant doit être observé sur une base régulière afin que le personnel enseignant soit en mesure d'évaluer de manière sommative l'atteinte de la compétence, la présence peut constituer une condition de réussite. Il importe de préciser que cette évaluation doit être mise en œuvre à partir des balises indiquées dans le devis ministériel. Les situations qui se prêtent à l'activation de cet article sont les suivantes :

- Le cours prévoit des heures de travail en atelier dans lesquelles il faut manœuvrer de manière sécuritaire différents types d'appareillage, ce qui nécessite des observations répétées afin de garantir la sécurité sur tous les appareils.
- Le cours prévoit des heures de travail en atelier où la personne étudiante doit démontrer le développement progressif de ses habiletés menant à une production finale.
- Le devis ministériel prévoit de manière explicite la pratique régulière d'une activité pendant les heures contacts.
- Le personnel enseignant n'est pas en mesure d'évaluer, en assurant la sécurité de la personne étudiante, sa pratique d'une activité physique. Celle-ci doit avoir démontré, pour certaines activités physiques et ce, sur une base régulière pendant les heures contacts, sa capacité à pratiquer l'activité de manière sécuritaire. La personne étudiante qui s'absente de manière régulière se verra refuser l'accès aux évaluations sommatives pratiques de cette activité physique pour assurer sa sécurité. Elle obtiendra la note zéro pour cette épreuve. Le département ou le conseiller ou la conseillère pédagogique à la Formation continue aura alors la responsabilité d'approuver cette pratique pour chacun des cours, puis de faire approuver les paramètres d'application par la Commission des études et de s'assurer que ces paramètres sont clairement explicités à l'intérieur de chacun des plans de cours afin que les personnes étudiantes en soient bien informées.

Plagiat ou fraude

Selon l'article 4.9.1 de la PIEA :

Le plagiat se définit comme l'acte de faire passer pour sien un contenu ou une production d'autrui sans en identifier la source. Commet un plagiat l'étudiant qui par exemple :

- Recopie un extrait d'un texte sans utiliser les normes de citation.
- S'approprie l'idée ou le texte d'un auteur en le paraphasant incorrectement ou en omettant d'utiliser les normes de citation.
- Utilise un concept, une image ou une musique sans en indiquer la source.

La fraude se définit comme l'acte de tromper dans le but d'en tirer un avantage personnel. Commet une fraude l'étudiant qui par exemple :

- Utilise un autre matériel que celui qui est autorisé, incluant le matériel qu'il a produit dans une évaluation pour un autre cours.
- Copie le travail ou les réponses d'examen d'une autre personne.
- Aide une autre personne à copier.
- Participe au vol, à la falsification de données, de document ou de matériel reliés à une évaluation ou à la justification d'une absence lors d'une évaluation (par exemple, n papier de médecin).

-
- Utilise de l'aide non permise pour réaliser un travail.

Tout plagiat, toute tentative de plagiat, toute collaboration à un plagiat entraîne la note zéro « 0 » pour l'évaluation en cause et doit faire l'objet d'un rapport écrit au Service des programmes et du développement pédagogique de la part de l'enseignante ou de l'enseignant.

Une récidive peut entraîner des mesures allant jusqu'au renvoi du Collège de l'étudiante ou de l'étudiant par la Direction des études. Pour en savoir plus sur la façon de citer ses sources dans un travail afin d'éviter le plagiat, consultez la [section suivante](#) du site internet de la [bibliothèque](#) du cégep Marie-Victorin.

MÉDIAGRAPHIE :

- Richard E. Silverman (2013), Git Pocket Guide. O'Reilly Media, Inc.
- Sonatype (2008), Maven: The Definitive Guide, O'Reilly Media, Inc.
- Nigel Poulton (2023), Docker Deep Dive, 2023 Edition, Nigel Poulton Ltd.
- Nigel Poulton & Pushkar Joglekar (2024), The Kubernetes Book, 2024 Edition, Nigel Poulton Ltd.
- Matt Butcher, Josh Dolitsky & Karen Chu (2021), Learning Helm, O'Reilly Media, Inc.
- Michael Kennedy (2023), Learning GitHub Actions, O'Reilly Media, Inc.
- Sonatype/OWASP (2024), documentation officielle de Trivy et SonarQube Community Edition (aquasecurity.github.io/trivy, docs.sonarsource.com)
- Robert C. Martin (2009), Clean Code: A Handbook of Agile Software Craftsmanship, Prentice Hall.
- Martin Fowler (2019), Refactoring: Improving the Design of Existing Code (2nd ed.), Addison-Wesley.